

Operationalizing & Automation

MISP Integration Workshop · CIRCL

Automation patterns

Data quality

Pitfalls to avoid

Q&A + pointers

Automation patterns

Event-driven pipelines — don't poll, react.

- **Workflows** — trigger actions on MISP events (on-publish, on-attribute-add): enrich, tag, block-list push, notify.
- **Webhooks** — push events out to external systems (SIEM, SOAR, chat) the moment they happen.
- Combine with the **pull model** (CDB list) we built for Wazuh — automation keeps the list fresh without manual exports.

Goal: an IOC published in MISP reaches your sensors with zero human steps.

Data quality

Garbage in → false positives out. Discipline matters more than volume.

- **Warninglists** — auto-flag known-good indicators (RFC1918, public DNS, CDNs, Alexa/Tranco top sites) so they never reach detection.
- **to_ids discipline** — only attributes meant for detection get `to_ids=1`; context-only IOCs stay out of exports.
- **Taxonomy consistency** — agree on TLP, confidence, and workflow tags across the org so filtering is predictable.

A small, curated feed beats a huge noisy one.

Pitfalls to avoid

The failure modes that bite teams in production.

- **False-positive IOCs** — warninglists + review before `to_ids=1`.
- **TLP leakage** — check TLP before any cross-org sync or integration push; `TLP:RED` /amber must not flow to broad block-lists.
- **IOC expiration** — set decay / first-seen-last-seen so stale indicators stop firing.
- **API auth-key hygiene** — scope keys per integration, rotate them, never commit them to git.

Q&A + pointers

Where to go next.

- **Docs** — <https://www.misp-project.org/documentation/>
- **MISP on GitHub** — <https://github.com/MISP/MISP>
- **PyMISP** — <https://github.com/MISP/PyMISP>
- **Training materials** — <https://github.com/MISP/misp-training>
- **This workshop's repo** — walkthrough, install guide, and lab OVAs.

Questions?